body { font-family: Helvetica, Arial, sans-serif; font-size: 10pt; line-height: 1.4; } h1 { font-size: 16pt; color: #003366; border-bottom: 2px solid #003366; margin-top: 20px; } h2 { font-size: 13pt; color: #004080; border-bottom: 1px solid #aaa; margin-top: 16px; } h3 { font-size: 11pt; color: #0059b3; margin-top: 12px; } table { border-collapse: collapse; width: 100%; margin: 8px 0; font-size: 9pt; } th { background: #003366; color: white; padding: 3px 5px; text-align: left; } td { border: 1px solid #999; padding: 2px 5px; } tr:nth-child(even) { background: #f5f5f5; } pre { background: #f0f0f0; border: 1px solid #ddd; border-left: 3px solid #003366; padding: 6px 8px; font-size: 8pt; } p { margin: 4px 0; } ul, ol { margin: 4px 0; padding-left: 18px; } li { margin: 1px 0; }

foxBMS 2 Complete Codebase Analysis & Custom BMS Adaptation Guide

Target MCU: TI TMS570LC4357 (ARM Cortex-R5)
RTOS: FreeRTOS Kernel V11.1.0
Documentation: v1.11.0
Generated: 2026-07-23

Table of Contents

1. Architecture Overview
2. Domain 1: System Startup & Main Execution Flow
3. Domain 2: RTOS Task Layer
4. Domain 3: Engine Layer
5. Domain 4: Application Layer - BMS Core
6. Domain 5: Algorithm Module & State Estimation
7. Domain 6: Driver Layer - AFE, Measurement, CAN
8. Domain 7: Driver Layer - Security, Diagnostics, Support
9. Domain 8: Portable Layer & HAL
10. Strategic Modification Approach for Custom BMS

1. Architecture Overview

```
src/
app/
main/           - System entry, startup, assertions
hal/            - Notification callbacks, startup hash
task/           - FreeRTOS integration, task creation, OS abstraction
engine/         - Database, diagnosis, system state machine, monitoring
application/    - BMS logic, SOA, balancing, algorithms, plausibility
driver/         - All peripheral drivers (AFE, CAN, SPI, I2C, etc.)
bootloader/       - Dual-bank firmware update
os/               - FreeRTOS kernel source
portable/         - Cortex-R5 port layer
version/          - Version info
```

The software is organized in three application layers atop FreeRTOS:

- **Driver Layer**: MCU-Wrapper abstraction providing CAN, UART, SPI, I2C, AFE, etc.
- **Engine Layer**: Database (producer/consumer), diagnosis (error handling), system state machine, task monitoring
- **Application Layer**: BMS state machine, SOA checking, balancing, plausibility, redundancy, algorithms

2. Domain 1: System Startup & Main Execution Flow

2.1 File Inventory

File	Path	Role
fstartup.c	src/app/main/fstartup.c	Entry point _c_int00()
fstartup.h	src/app/main/include/fstartup.h	Declaration
main.c	src/app/main/main.c	main() function
main.h	src/app/main/include/main.h	Declaration
fassert.c	src/app/main/fassert.c	Assertion storage
fassert.h	src/app/main/include/fassert.h	FAS_ASSERT() macro
app-hl_notification.c	src/app/hal/app-hl_notification.c	HAL ISR callbacks
app-startup.hash	src/app/hal/app-startup.hash	MD5 of HALCoGen config
app_build_cfg.h	src/app/main/include/app_build_cfg.h	Build configuration
master_info.c	src/app/engine/hw_info/master_info.c	Reset source storage
os.c	src/app/task/os/os.c	OS boot state machine
ftask.c	src/app/task/ftask/ftask.c	Task body definitions
ftask_cfg.c	src/app/task/config/ftask_cfg.c	User code hooks

2.2 Complete Power-On Flow

```
POWER-ON RESET
-> CPU fetches from 0x00000000 (bootloader vector table)
-> Bootloader main(): CAN FSM + Boot FSM
-> On timeout or explicit RUN command:
-> BOOT_IsProgramAvailableAndValidated() (CRC64 verification)
-> BOOT_JumpInToLastFlashedProgram()
-> function pointer call to 0x0001FFE0 (app vector table)

APPLICATION STARTUP:
-> CPU loads SP from [0x0001FFE0], PC from [0x0001FFE4] = _c_int00()
-> _c_int00() in fstartup.c:
1. _coreInitRegisters_() - init core regs for CCM
2. _coreInitStackPointer_() - init SP for all modes
3. STU_GetResetSourceWithoutFlagReset()
4. Switch on reset source:
POWERON_RESET: _memInit_() + fallthrough
DEBUG_RESET/EXT_RESET: systemInit(), vimInit(), esmInit()
CPU0_RESET: event bus export only
5. _mpuInit_() - Memory Protection Unit
6. __TI_auto_init() - C runtime (.data, .bss, constructors)
7. main()

main() in main.c:
Steps 1-17: Hardware peripheral init (mux, GIO, SPI, ADC, HET,
eTPWM, CRC, I2C, DMA, UART, PWM, DIAG, MATH, ETH)
Then: OS_InitializeOperatingSystem()
1. OS_InitializeScheduler()
2. FTSK_CreateQueues()
3. FTSK_CreateTasks()
4. _enable_IRQ_interrupt_()
5. OS_StartScheduler() -> FreeRTOS vTaskStartScheduler()
```

2.3 Boot State Machine

```
OS_OFF -> OS_INITIALIZE_SCHEDULER -> OS_CREATE_QUEUES -> OS_CREATE_TASKS
-> OS_INIT_PRE_OS -> [vTaskStartScheduler()]
-> OS_SCHEDULER_RUNNING (engine task)
-> OS_ENGINE_RUNNING (engine task after DB init)
-> OS_PRE_CYCLIC_INITIALIZATION_HAS_FINISHED (1ms task)
-> OS_SYSTEM_RUNNING (algorithm 100ms task, last)
```

2.4 Key Functions

Function	File	Description
_c_int00()	fstartup.c	System reset handler
STU_GetResetSourceWithoutFlagReset()	fstartup.c (static)	Read SYS_EXCEPTION without clearing
main()	main.c	17-step hardware init + OS start
FAS_ASSERT(x)	fassert.h (macro)	Assert with infinite loop or debug halt
FAS_StoreAssertLocation()	fassert.c	Records PC + line number
esmGroup3Notification()	app-hl_notification.c	Fatal ESM error -> infinite loop
MINFO_SetResetSource()	master_info.c	Store reset cause

2.5 Custom MCU Adaptation Points

What to Modify	File	Change Required
Replace HALCoGen init	main.c steps 2-8, 10-11	Replace with your MCU's HAL
CPU startup	fstartup.c	Replace _c_int00() with your MCU reset handler
MPU/FPU config	fstartup.c	Replace _mpuInit_(), _coreInitRegisters_()
Startup hash	app-startup.hash	Replace with hash of your HAL config
Linker script	app.cmd, app_hex.cmd	Redefine memory map for your MCU
Interrupt vectors	app/main/include/	Define your MCU's vector table
Assert behavior	fassert.h	Change FAS_ASSERT_LEVEL or FAS_InfiniteLoop()
Notification callbacks	app-hl_notification.c	Implement your MCU's peripheral ISR callbacks

3. Domain 2: RTOS Task Layer (FreeRTOS)

3.1 File Inventory

File	Path	Role
FreeRTOSConfig.h	src/os/freertos/freertos/include/	RTOS config
ftask.c	src/app/task/ftask/ftask.c	Task body implementations
ftask.h	src/app/task/ftask/ftask.h	Task/queue declarations
ftask_freertos.c	src/app/task/ftask/freertos/ftask_freertos.c	Queue + task creation
ftask_cfg.c	src/app/task/config/ftask_cfg.c	User-code hook implementations
ftask_cfg.h	src/app/task/config/ftask_cfg.h	Task definitions
os.c	src/app/task/os/os.c	OS wrapper (timer, boot)
os.h	src/app/task/os/os.h	OS types
os_freertos.c	src/app/task/os/freertos/os_freertos.c	FreeRTOS implementation
timer.c	src/app/task/timer/timer.c	Software timer wrapper
timer.h	src/app/task/timer/timer.h	Timer API

3.2 FreeRTOS Configuration

Parameter	Value	Impact
configTICK_RATE_HZ	1000	1ms tick
configCPU_CLOCK_HZ	HALCoGen-defined	300 MHz typical
configTOTAL_HEAP_SIZE	40960 bytes	40KB heap
configMAX_PRIORITIES	11	Levels 0-10
configUSE_PREEMPTION	1	Preemptive
configUSE_TIME_SLICING	0	No round-robin
configUSE_TIMERS	1	Software timers enabled
configCHECK_FOR_STACK_OVERFLOW	2	Method 2
configUSE_MUTEXES	0	No mutexes (uses critical sections)

3.3 Complete Task Table

Task Name	Creator	Priority	Stack	Period	Phase(ms)
"TaskEngine"	FTSK_CreateTaskEngine	REAL_TIME(9)	1024B	1ms polling	0
"TaskCyclic1ms"	FTSK_CreateTaskCyclic1ms	VERY_HIGH(7)	1024B	1ms	0
"TaskCyclic10ms"	FTSK_CreateTaskCyclic10ms	HIGH(5)	5120B	10ms	2
"TaskCyclic100ms"	FTSK_CreateTaskCyclic100ms	ABOVE_NORMAL(4)	1024B	100ms	56
"TaskCyclicAlgorithm100ms"	FTSK_CreateTaskCyclicAlgorithm100ms	NORMAL(3)	1024B	100ms	64

Task Name	Creator	Priority	Stack	Period	Phase(ms)
"TaskI2c"	FTSK_CreateTaskI2c	HIGH(5)	2048B	continuous	0
"TaskAfe" (opt)	FTSK_CreateTaskAfe	ABOVE_HIGH(6)	4096B	continuous	0
"TaskUart" (opt)	FTSK_CreateTaskUart	NORMAL(3)	1024B	blocked	0
"TaskEmac" (opt)	FTSK_CreateTaskEmac	ABOVE_NORMAL(4)	2048B	continuous	10

3.4 Task Execution Details

Engine Task (highest priority, never blocks):

```
FTSK_CreateTaskEngine():
1. OS_MarkTaskAsRequiringFpuContext()
2. os_boot = OS_SCHEDULER_RUNNING
3. DATA_Initialize(), FRAM_Initialize(), SYSM_Initialize()
4. os_boot = OS_ENGINE_RUNNING
5. while(1) { DATA_Task(); SYSM_CheckNotifications(); }
```

1ms Task (cyclic via `vTaskDelayUntil`):

```
FTSK_RunUserCodeCyclic1ms():
OS_IncrementTimer() - cascading timer (ms -> days)
DIAG_UpdateFlags()
MEAS_Control() [if FSM mode]
CAN_ReadRxBuffer()
```

10ms Task (heavy lifting):

```
FTSK_RunUserCodeCyclic10ms():
SYSM_UpdateFramData(), SYS_Trigger(), ILCK_Trigger(),
ADC_Control(), SPS_Ctrl(), CAN_MainFunction(),
SOF_Calculation(), ALGO_MonitorExecutionTime(),
SBC_Trigger(), BMS_Trigger()
[every 50ms: MRC_ValidateAfeMeasurement(), MRC_ValidatePackMeasurement()]
```

100ms Task:

```
FTSK_RunUserCodeCyclic100ms():
BAL_Trigger(), IMD_Trigger(), LED_Trigger(),
MINFO_CheckSupplyVoltageClamp30c()
[every 1s: SE_RunStateEstimations()]
```

3.5 Queue Definitions

Queue Handle	Length	Item Size	Purpose
ftsk_databaseQueue	1	DATA_QUEUE_MESSAGE_s	Database message passing
ftsk_imdCanDataQueue	5	CAN_BUFFER_ELEMENT_s	IMD CAN data
ftsk_canRxQueue	50	CAN_BUFFER_ELEMENT_s	CAN received messages

Queue Handle	Length	Item Size	Purpose
ftsk_canTxUnsentMessagesQueue	5	CAN_BUFFER_ELEMENT_s	CAN TX retry buffer
ftsk_afeRequestQueue	1	AFE_REQUEST_e	AFE driver requests
ftsk_rtcSetTimeQueue	1	RTC_TIME_DATA_s	RTC time setting
ftsk_afeToI2cQueue	1	AFE_I2C_QUEUE_s	I2C TX via AFE slave
ftsk_afeFromI2cQueue	1	AFE_I2C_QUEUE_s	I2C RX via AFE slave
ftsk_uartRxQueue	1024	uint8_t	UART received bytes

3.6 Custom RTOS Adaptation Points

What to Modify	File	Change Required
Port to new MCU	src/os/	Replace FreeRTOS port for your architecture
Tick rate	FreeRTOSConfig.h	Set configCPU_CLOCK_HZ to your MCU speed
Task priorities/periods	ftask_cfg.h	Adjust for your timing requirements
Stack sizes	ftask_cfg.h	Adjust based on your stack usage
Task functions	ftask_cfg.c	Replace user-code hooks
Queue configuration	ftask_freertos.c	Modify queue sizes/types
Critical sections	os_freertos.c	Replace with your MCU's IRQ disable

4. Domain 3: Engine Layer

4.1 Database Module

Files: `database.c`, `database.h`, `database_helper.c`, `database_helper.h`, `database_cfg.c`, `database_cfg.h`

Architecture: Producer/Consumer via RTOS Queue

The database uses a single-element RTOS message queue (`ftsk_databaseQueue`). The `DATA_Task()` (Engine task) is the **sole writer** to all database entries. All other tasks send read/write requests via the queue.

Key Data Structures

```
typedef struct {
DATA_BLOCK_ID_e uniqueId;    // From enum (41 IDs)
uint32_t timestamp;          // Last update tick
uint32_t previousTimestamp;  // Previous update tick
} DATA_BLOCK_HEADER_s;

typedef struct {
void *pDatabaseEntry;  // Pointer to DB entry
uint32_t dataLength;   // Size in bytes
} DATA_BASE_s;
```

```
typedef struct {
DATA_BLOCK_ACCESS_TYPE_e accessType;  // READ or WRITE
void *pDatabaseEntry[DATA_MAX_ENTRIES_PER_ACCESS]; // 4 pointers
} DATA_QUEUE_MESSAGE_s;
```

All 41 Database Block IDs

ID	Struct	Content
DATA_BLOCK_ID_CELL_VOLTAGE	DATA_BLOCK_CELL_VOLTAGE_s	Validated cell voltages
DATA_BLOCK_ID_CELL_VOLTAGE_BASE	same	Base AFE measurement
DATA_BLOCK_ID_CELL_VOLTAGE_REDUNDANCY0	same	Redundancy AFE measurement
DATA_BLOCK_ID_CELL_TEMPERATURE	DATA_BLOCK_CELL_TEMPERATURE_s	Validated temperatures
DATA_BLOCK_ID_CELL_TEMPERATURE_BASE	same	Base AFE measurement
DATA_BLOCK_ID_CELL_TEMPERATURE_REDUNDANCY0	same	Redundancy measurement
DATA_BLOCK_ID_CURRENT	DATA_BLOCK_CURRENT_s	Per-string current (mA)
DATA_BLOCK_ID_POWER	DATA_BLOCK_POWER_s	Per-string power (W)
DATA_BLOCK_ID_CURRENT_COUNTER	DATA_BLOCK_CURRENT_COUNTER_s	Coulomb counter (As)
DATA_BLOCK_ID_ENERGY_COUNTER	DATA_BLOCK_ENERGY_COUNTER_s	Energy counter (Wh)
DATA_BLOCK_ID_SYSTEM_VOLTAGE_1/2/3	DATA_BLOCK_SYSTEM_VOLTAGE_s	Current sensor voltages
DATA_BLOCK_ID_SOC	DATA_BLOCK_SOC_s	SOC per string
DATA_BLOCK_ID_SOE	DATA_BLOCK_SOE_s	SOE per string
DATA_BLOCK_ID_SOH	DATA_BLOCK_SOH_s	SOH per string
DATA_BLOCK_ID_SOF	DATA_BLOCK_SOF_s	Recommended current limits
DATA_BLOCK_ID_MIN_MAX	DATA_BLOCK_MIN_MAX_s	Min/max cell V & T
DATA_BLOCK_ID_PACK_VALUES	DATA_BLOCK_PACK_VALUES_s	Pack-level values
DATA_BLOCK_ID_ERROR_STATE	DATA_BLOCK_ERROR_STATE_s	~55 error flags
DATA_BLOCK_ID_MOL_FLAG	DATA_BLOCK_MOL_FLAG_s	Max Operating Limit flags
DATA_BLOCK_ID_RSL_FLAG	DATA_BLOCK_RSL_FLAG_s	Rec. Safety Limit flags
DATA_BLOCK_ID_MSL_FLAG	DATA_BLOCK_MSL_FLAG_s	Max Safety Limit flags
DATA_BLOCK_ID_BALANCING_CONTROL	DATA_BLOCK_BALANCING_CONTROL_s	Balancing commands
DATA_BLOCK_ID_CONTACTOR_FEEDBACK	DATA_BLOCK_CONTACTOR_FEEDBACK_s	Contactor status
DATA_BLOCK_ID_INSULATION	DATA_BLOCK_INSULATION_s	IMD data
DATA_BLOCK_ID_INTERLOCK_FEEDBACK	DATA_BLOCK_INTERLOCK_FEEDBACK_s	HVIL status

ID	Struct	Content
DATA_BLOCK_ID_STATE_REQUEST	DATA_BLOCK_STATE_REQUEST_s	CAN state requests
DATA_BLOCK_ID_SYSTEM_STATE	DATA_BLOCK_SYSTEM_STATE_s	BMS CAN state
DATA_BLOCK_ID_ADC_VOLTAGE	DATA_BLOCK_ADC_VOLTAGE_s	MCU internal ADC
DATA_BLOCK_ID_MOVING_AVERAGE	DATA_BLOCK_MOVING_AVERAGE_s	Current/power averages
DATA_BLOCK_ID_OPEN_WIRE_BASE/_REDUNDANCY0	DATA_BLOCK_OPEN_WIRE_s	Open wire detection
DATA_BLOCK_ID_HTSEN	DATA_BLOCK_HTSEN_s	Humidity/temperature
DATA_BLOCK_ID_AEROSOL_SENSOR	DATA_BLOCK_AEROSOL_SENSOR_s	Aerosol detection
DATA_BLOCK_ID_PHY	DATA_BLOCK_PHY_s	Ethernet PHY status

Key Database Functions

Function	Description
DATA_Initialize()	Build lookup table, zero all entries
DATA_Task()	Poll queue, call DATA_IterateOverDatabaseEntries()
DATA_WRITE_DATA(&s1, &s2, ...)	Macro: write 1-4 blocks atomically via queue
DATA_READ_DATA(&s1, &s2, ...)	Macro: read 1-4 blocks atomically via queue
DATA_ExecuteDataBist()	Self-test: write/read dummy block

4.2 Diagnosis (DIAG) Module

Files: `diag.c`, `diag.h`, `diag_cfg.c`, `diag_cfg.h`, `diag_cbs_*.c` (21 files)

85 diagnosis IDs covering: system monitoring, AFE SPI/CONFIG/MUX, CAN timing, current sensor, plausibility (V/T/I), SOA limits (MSL/RSL/MOL), contactor feedback, precharge, SBC, interlock, insulation, RTC, FRAM, aerosol, supply voltage.

Error levels: `DIAG_FATAL_ERROR` (opens contactors + CAN message), `DIAG_WARNING` (flags only), `DIAG_INFO`

Debouncing: occurrenceCounter per ID per string. Event must occur N times (threshold) before error declared. Recovery decrements on OK events.

Function	Description
DIAG_Initialize(DIAG_DEV_s*)	Build id2ch lookup, configure enable flags
DIAG_Handler(id, event, impact, data)	Central state machine with debounce
DIAG_CheckEvent(cond, id, impact, data)	Wrapper: OK if STD_OK, else NOT_OK
DIAG_IsAnyFatalErrorSet()	True if any fatal error threshold exceeded
DIAG_UpdateFlags()	Write local error flags to database

4.3 System Module

Files: `sys.c`, `sys.h`, `sys_cfg.c`, `sys_cfg.h`, `reset.c`, `reset.h`

State Machine

```
UNINITIALIZED -> INITIALIZATION -> PRE_RUNNING -> RUNNING
|                  |
v                  v
ERROR              ERROR  (latching)
```

INITIALIZATION substates (25): Check deep discharge -> Init SBC -> Init CAN -> Init RTC -> BIST -> Send boot CAN message -> PRE_RUNNING PRE_RUNNING substates: Init interlock -> Init balancing -> First measurement -> Current sensor check -> Init IMD -> Init BMS -> RUNNING

4.4 System Monitoring

Files: `sys_mon.c`, `sys_mon.h`, `sys_mon_cfg.c`, `sys_mon_cfg.h`

Monitors 5 tasks via `SYSM_Notify(ENTER/EXIT)`. Timing violations trigger `DIAG_ID_SYSTEM_MONITORING` (FATAL_ERROR). Violations recorded to FRAM.

4.5 Custom Engine Adaptation Points

What to Modify	File	Change
Database entries	database_cfg.c/h	Add/remove DB blocks for your sensors
Diagnosis IDs	diag_cfg.c/h	Add/remove error conditions
Error thresholds	diag_cfg.c	Tune debounce for your system
System state machine	sys_cfg.c	Modify init sequence for your HW
Task monitoring	sys_mon_cfg.h	Adjust timing tolerances

5. Domain 4: Application Layer - BMS Core

5.1 BMS State Machine

Files: `bms.c`, `bms.h`, `bms_cfg.h`

States (14)

```
UNINITIALIZED -> INITIALIZATION -> INITIALIZED -> IDLE
-> [CAN: STANDBY] -> OPEN_CONTACTORS -> STANDBY
-> [CAN: NORMAL or CHARGE] -> PRECHARGE
-> NORMAL (discharge) or CHARGE
-> [Fatal error at any time] -> ERROR
```

Key Functions

Function	Description
BMS_Trigger()	Main state machine, called every 10ms
BMS_SetStateRequest(req)	Set CAN-driven state request
BMS_IsBatterySystemStateOkay()	Central safety gate: checks all fatal errors, delay timer
BMS_IsContactorFeedbackValid()	Cross-checks commanded vs actual contactor state
BMS_CheckOpenSenseWire()	Iterates all voltage sense wires
BMS_MonitorPrechargeProcess()	Monitors current + voltage thresholds

Function	Description
BMS_GetClosestString()	Selects next string to close in NORMAL state

Configuration Parameters

Parameter	Default	Description
BMS_PRECHARGE_TRIES	3	Max precharge retries
BMS_PRECHARGE_VOLTAGE_THRESHOLD_mV	1000 mV	Voltage diff for precharge success
BMS_PRECHARGE_CURRENT_THRESHOLD_mA	50 mA	Current threshold for precharge success
BMS_MAXIMUM_PRECHARGE_DURATION_ms	2000 ms	Max precharge time
BMS_NEXT_STRING_VOLTAGE_LIMIT_MV	3000 mV	Max voltage diff to close next string

5.2 SOA (Safe Operating Area)

Files: `soa.c`, `soa.h`, `soa_cfg.c`, `soa_cfg.h`

Three severity levels: **MOL** (info), **RSL** (warning), **MSL** (fatal).

Function	Description
SOA_CheckVoltages(minMax)	Overvoltage/undervoltage MSL/RSL/MOL + deep discharge
SOA_CheckTemperatures(minMax, pack)	Charge/discharge temperature limits
SOA_CheckCurrent(pack)	Per-string, per-cell, per-pack current limits

5.3 Balancing Module

Files: `bal.c`, `bal.h`, `bal_strategy_voltage.c`, `bal_strategy_history.c`, `bal_strategy_none.c`, `bal_cfg.c`, `bal_cfg.h`

Strategies: Voltage-based (cells above min+threshold get balanced), History-based (SOC delta charge), None

Parameter	Default	Description
BAL_DEFAULT_THRESHOLD_mV	200 mV	Voltage threshold
BAL_HYSTERESIS_mV	200 mV	Hysteresis after completion
BAL_LOWER_VOLTAGE_LIMIT_mV	2000 mV	Stop below this voltage
SLV_BALANCING_RESISTANCE_ohm	100 ohm	Balancing resistor

5.4 Plausibility Module

Function	Tolerance	Description
PL_CheckCellVoltage(base, red0, pAvg)	10 mV	Cross-check -> average
PL_CheckCellTemperature(base, red0, pAvg)	50 dK	Cross-check -> average
PL_CheckStringVoltage(afe, sensor)	3000 mV	AFE vs current sensor
PL_CheckVoltageSpread(voltages, minMax)	300 mV	Cell deviation from avg
PL_CheckTemperatureSpread(temps, minMax)	100 dK	Sensor deviation from avg

5.5 Redundancy Module

Files: `redundancy.c`, `redundancy.h`

Called every 50ms. Validates base + redundancy0 measurements via Plausibility, handles timeouts, writes validated output: - Both updated -> plausibility check -> use average - Only one source -> use that (other timed out) - Neither -> no data

5.6 Custom Application Adaptation Points

What to Modify	File	Change
Cell type parameters	battery_cell_cfg.c/h	Set your cell's V/T/I limits, OCV curves, capacity
System topology	battery_system_cfg.h	Set strings, modules, cells, temp sensors
BMS precharge params	bms_cfg.h	Adjust for your precharge circuit
SOA limits	soa_cfg.c/h	Set MOL/RSL/MSL for your cell chemistry
Balancing strategy	wscript	Select voltage, history, or none
Plausibility tolerances	plausibility_cfg.h	Set measurement cross-check tolerances

6. Domain 5: Algorithm Module & State Estimation

6.1 Algorithm Scheduler

Files: `algorithm.c`, `algorithm.h`, `algorithm_cfg.c`, `algorithm_cfg.h`

Table-driven scheduler (100ms tick). Algorithms registered in `algo_algorithms[]` table. Default: only `MovingAverage`.

6.2 State Estimation API

Every SOC/SOE/SOH/SOF must provide these functions (build-time selected):

Function	Purpose
SE_InitializeStateOfCharge(soc, ccPresent, string)	Init from FRAM

Function	Purpose
SE_CalculateStateOfCharge(soc)	Periodic SOC update
SE_GetStateOfChargeFromVoltage(mV) -> float	OCV-to-SOC lookup
SE_InitializeStateOfEnergy(soe, ecPresent, string)	Init from FRAM
SE_CalculateStateOfEnergy(soe)	Periodic SOE update
SE_InitializeStateOfHealth(soh, string)	Init SOH
SE_CalculateStateOfHealth(soh)	Periodic SOH update

6.3 SOC Implementations

Variant	Method
counting	Coulomb counting + OCV recalibration at rest. Integrates current with timestamps or CC sensor. FRAM-persistent.
lookup-table	Pure OCV lookup, no current integration
debug	Stub: always 50%
none	Stub: always 0%

OCV Lookup Table: 85 entries, 4123mV (100%) to 2716mV (1%), linear interpolation.

6.4 SOE and SOH

SOE: counting (energy integration + OCV recalibration), debug, none
SOH: debug only (hardcoded 50%) - **no actual capacity fade estimation in open-source**
SOF: trapezoid derating based on voltage and temperature curves

6.5 Custom Algorithm Adaptation Points

What to Modify	File	Change
OCV lookup table	battery_cell_cfg.c	Replace with your cell's OCV curve
Cell capacity	battery_cell_cfg.h	Set BC_CAPACITY_mAh
SOH algorithm	soh/<variant>/	Implement capacity fade / resistance estimation
SOC algorithm	soc/<variant>/	Implement Kalman filter or enhanced counting
SOF derating	sof/trapezoid/sof_trapezoid_cfg.c	Set temperature/voltage cutoff curves

7. Domain 6: Driver Layer - AFE, Measurement, CAN, Contactor, SPI, I2C

7.1 Analog Front-End (AFE)

Files: `afe/afe.c`, `afe.h`, plus per-chip subdirectories

Supported AFE Chips

Chip	Cells	Interface
LTC6804-1	12	SPI daisy-chain
LTC6811-1	12	SPI daisy-chain
LTC6812-1	15	SPI daisy-chain
LTC6813-1	18	SPI daisy-chain
LTC6806	12 + temp	SPI daisy-chain
MAX1785x	12-14	UART daisy-chain
ADES1830	18	SPI
MC33775A	14	SPI transformer-isolated
Debug Default	stub	Testing

AFE API (per-chip implementation)

Function	Description
AFE_Initialize()	Chip-specific init
AFE_WakeUp()	Wake from sleep
AFE_StartMeasurement()	Trigger ADC conversion
AFE_ReadVoltages()	Read cell voltages
AFE_ReadTemperatures()	Read temperature sensors
AFE_StartOpenWireDetection()	Begin open wire test
AFE_ReadOpenWireStatus()	Get open wire results
AFE_SetBalanceSwitches()	Control balancing MOSFETs

7.2 Measurement Module

Files: `meas.c`, `meas.h`

State machine: IDLE -> MEASURE -> WAIT -> READ. Two modes: FSM (1ms task-driven) and NO_FSM (continuous loop).

7.3 CAN Module

Files: `can/can.c`, `can.h`, `can_cfg.c`, `can_cfg.h`

HALCoGen DCAN driver with interrupt-driven RX. CAN RX ISR -> queue -> `CAN_MainFunction()` (10ms task). Custom protocol (not CANopen). DBC file available.

Message categories: State requests, system state broadcasts, cell V/T broadcasts, fatal error messages, debug boot messages.

7.4 Contactor Driver

Files: `contactor/contactor.c`, `contactor.h`, `contactor_cfg.h`

State machine: UNINITIALIZED -> INIT -> IDLE -> CLOSE_MINUS -> CHECK -> CLOSE_PRECHARGE -> ... -> RUNNING. Weld detection via feedback check with timeout.

7.5 Custom Driver Adaptation Points

What to Modify	File	Change
AFE chip selection	afe/wscript	Select via FOXBMS_AFE_DRIVER_TYPE
AFE daisy-chain config	afe/<chip>/	Modify cell count, addressing
CAN protocol	can_cfg.c/h	Define your message IDs and DBC
CAN bitrate	HL_can.h	Set via HALCoGen
Contactor GPIO mapping	contactor_cfg.h	Map to your GPIO pins
SPI frequency	HL_spi.h	Set via HALCoGen
I2C frequency	HL_i2c.h	Set via HALCoGen

8. Domain 7: Driver Layer - Security, Diagnostics, Support

8.1 SBC (System Basis Chip) - NXP FS85xx/FS8x

Files: `sbc.c`, `sbc.h`, `nxpfs85xx.c`, `nxpfs85xx.h`, `sbc_fs8x.c`, `sbc_fs8x.h`, `sbc_fs8x_communication.c`

State machine: OFF -> INIT -> RUNNING -> FAIL. Features: fail-safe outputs (FS0B/FS1B), window watchdog, supply monitoring, reset generation.

Diagnosis: `DIAG_ID_SBC_FIN_ERROR`, `DIAG_ID_SBC_RSTB_ERROR`

8.2 IMD (Insulation Monitoring)

Files: `imd.c`, `imd.h`, `bender_ir155.c`, `bender_iso165c.c`, `none_none.c`

Supported devices: Bender IR155 (CAN), Bender iso165c (CAN), None (dummy). State machine: OFF -> INIT -> WAKING_UP -> RUNNING.

Diagnosis: `DIAG_ID_INSULATION_MEASUREMENT_VALID`, `DIAG_ID_LOW_INSULATION_RESISTANCE_ERROR/WARNING`, `DIAG_ID_INSULATION_GROUND_ERROR`

8.3 Other Drivers Summary

Module	Files	Key Functions
Interlock (HVIL)	interlock.c/h	ILCK_Trigger(), ILCK_Enable(), ILCK_Disable()
RTC	rtc.c/h	RTC_Trigger(), RTC_GetTime(), RTC_SetTime()
FRAM	fram.c/h, fram_cfg.c/h	FRAM_ReadData(), FRAM_WriteData(), 7 persistent blocks
IO	io.c/h	GPIO abstraction (digital in/out)
LED	led.c/h	Status LED patterns
PEX	pex.c/h, pex_cfg.c	NXP PCA9539 I2C port expander (3 chips)
CRC	crc.c/h	CRC32/CRC64 computation
HTSENSOR	htsensor.c/h	I2C humidity/temperature sensor
DMA	dma.c/h, dma_cfg.c	14 channels: SPI(10), I2C(4), UART(1)

Module	Files	Key Functions
PWM	pwm.c/h	PWM generation
UART	uart.c/h	Debug serial with XON/XOFF flow control
ADC	adc.c/h	MCU internal ADC for supply monitoring
SPS	sps.c/h	Serial peripheral switch (contactor feedback)
Temperature Sensor (TSI)	ts.c/h	NTC thermistor via LUT + polynomial
PHY	phy.c/h	Ethernet PHY (DP838xx, KSZ8081)

8.4 Custom Security/Support Adaptation

What to Modify	File	Change
SBC model	sbc/	Replace NXP FS85xx with your SBC
Watchdog timing	sbc_fs8x_communication.c	Set watchdog period
IMD model	imd/wscript	Select Bender IR155/iso165c/none
FRAM chip	fram.c	Replace SPI FRAM with your NVM
FRAM blocks	fram_cfg.c/h	Add/remove persistent data
RTC chip	rtc.c	Replace with your I2C RTC
Port expander	pex_cfg.c	Set I2C addresses, pin mappings

9. Domain 8: Portable Layer & HAL (Cortex-R5, MCU-Specific)

9.1 Portable Layer

Files: `src/portable/cortex-r5/` (port.c, portmacro.h, portasm.asm)

FreeRTOS port for Cortex-R5: `vPortStartFirstTask()`, `xPortPendSVHandler()`, `vPortSVCHandler()`, assembly yield and SWI handlers.

9.2 HALCoGen Integration

Generated code by TI HALCoGen (all `HL_` prefixed). DO NOT hand-modify:

File	Content
HL_system.c/h	systemInit(), PLL config, clock tree
HL_sys_core.h	_c_int00(), _coreInitRegisters_(), _mpuInit_()
HL_sys_vim.h/c	Vectored Interrupt Manager (VIC)
HL_esm.h/c	Error Signaling Module
HL_adc.h/c, HL_can.h/c, HL_spi.h/c, HL_i2c.h/c	Peripheral drivers
HL_gio.h/c, HL_het.h/c, HL_etpwm.h/c	GPIO, HET timer, eTPWM
HL_sci.h/c, HL_crc.h/c, HL_dma.h/c	SCI/UART, CRC, DMA

9.3 Bootloader

Files: `src/bootloader/`

Dual-Bank Flash Layout

```
0x00000000: Vector table (bootloader)
0x00000020: Bootloader code
0x00018000: Program info (CRC, length, magic)
0x0001FFE0: Application vector table [jump target]
0x00020000: Application code
```

Bootloader State Machine

```
WAIT -> [timeout] -> RUN
WAIT -> [CAN: LOAD] -> LOAD (flash programming)
LOAD -> [finished] -> RUN
RUN -> [app validated by CRC64] -> JUMP TO 0x0001FFE0
```

9.4 Custom Portable/HAL Adaptation

What to Modify	File	Change
Entire MCU port	src/portable/	Replace cortex-r5 with your arch
HALCoGen project	config/halcogen/	Regenerate for your MCU
Startup hash	app-startup.hash	New MD5 of your HAL config
Bootloader	src/bootloader/	Rewrite for your flash geometry
Linker script	app.cmd	Define your flash/RAM map
Interrupt controller	HL_sys_vim.h	Replace VIC with NVIC (Cortex-M)

10. Strategic Modification Approach for Custom BMS

Phase 1: MCU Port (Hardest - Do First)

1. **Replace** `src/portable/` with your MCU's FreeRTOS port
2. **Replace** HALCoGen-generated `HL_*` files with your MCU's HAL
3. **Rewrite** `fstartup.c` reset handler for your MCU
4. **Rewrite** `app.cmd` linker script for your memory map
5. **Replace** `app-hl_notification.c` with your MCU's ISR callbacks
6. **Rewrite** bootloader for your flash geometry

Phase 2: Driver Port

1. **SPI/I2C/UART**: Replace HALCoGen calls with your MCU's HAL
2. **AFE**: Select your chip in wscript, configure daisy-chain
3. **CAN**: Define your protocol, message IDs, DBC file
4. **SBC/IMD/RTC/FRAM**: Replace with your specific chips
5. **ADC/IO/LED/PWM**: Map to your GPIO/peripheral pins

Phase 3: Application Configuration

1. **battery_cell_cfg.h**: Set your cell's V/T/I limits, OCV curves
2. **battery_system_cfg.h**: Define your pack topology
3. **bms_cfg.h**: Tune precharge, contactor, timing parameters
4. **soa_cfg.h**: Set MOL/RSL/MSL limits
5. **bal_cfg.h**: Set balancing strategy and thresholds

Phase 4: Algorithm Tuning

1. **OCV tables**: Load your cell's OCV-SOC and OCV-SOE curves
2. **Capacity/Energy**: Set your cell's Ah and Wh ratings
3. **SOC algorithm**: Select counting or lookup-table; implement Kalman if desired
4. **SOH algorithm**: Implement capacity fade and resistance estimation
5. **SOF curves**: Set temperature/voltage derating curves

Phase 5: Verification

1. Run `MATH_StartupSelfTest()` for math integrity
2. Run `DATA_ExecuteDataBist()` for database integrity
3. Test fault injection via `DIAG_Handler()` callbacks
4. Verify task timing via `SYSM_CheckNotifications()`
5. End-to-end: CAN commands -> BMS state transitions -> contactor control

Key Architectural Insight

The **Database module** (`DATA_*`) is the backbone of all inter-task communication. Every sensor reading, calculated value, and error flag flows through it. The **Engine task** is the sole writer; all other tasks are readers via the RTOS message queue. This producer/consumer pattern is the single most critical architectural invariant to preserve when porting.

Most files to modify for new hardware: `battery_cell_cfg.h/.c`, `battery_system_cfg.h`, `bms_cfg.h`, `soa_cfg.h`, `plausibility_cfg.h`, and the `driver/afe/<your chip>/` directory.

Least files to modify: The BMS logic (states, SOA, balancing, plausibility, redundancy) in `application/` and `engine/` is largely hardware-independent and can be preserved with only configuration changes.

End of Report